Databricks Milestone 2 – 90 Scenario-Based MCQs

30 Easy + 30 Medium + 30 Hard • Based on the uploaded Milestone 2 Preparation Handbook

Question style follows the handbook emphasis: concept recall, code interpretation, basic application, and scenario reasoning. Hard questions focus on stretch areas such as streaming, JDBC and concurrency.

Easy – 30 Questions

Q1. [Java / JVM] A trainee has App.java and wants to compile it before running it. Which component/tool is used to compile the source into bytecode?

A. JVM
B. javac from the JDK
C. JRE only
D. Garbage Collector

Answer: B. javac from the JDK

Q2. [Java Collections] A store must keep customer IDs unique, and order/indexed access is not required. Which collection best fits?

A. ArrayList
B. HashSet
C. HashMap
D. ListIterator

Answer: B. HashSet

Q3. [Java Collections] A support application frequently looks up a customer name using a numeric customer ID. Which structure is most suitable?

A. HashMap
B. HashSet
C. ArrayList
D. CopyOnWriteArrayList

Answer: A. HashMap

Q4. [Java Generics] A method accepts a list containing Integer or another Number subtype and only needs to read values as Number. Which type is appropriate?

A. List
B. List
C. List only
D. List

Answer: A. List

Q5. [Java Operators] A program evaluates `user != null && user.isActive()`. If user is null, what happens to the right side?

A. It is evaluated and throws NullPointerException
B. It is skipped because && short-circuits
C. It runs only after compilation
D. It is evaluated twice

Answer: B. It is skipped because && short-circuits

Q6. [Java OOP] A developer writes `Product(String name) { this.name = name; }`. What is this construct?

A. Static method
B. Constructor
C. Overloaded field
D. Interface

Answer: B. Constructor

Q7. [Java OOP] A parent reference points to a Dog object, and Dog overrides sound(). Which implementation runs when `a.sound()` is called?

A. Animal's implementation always
B. Dog's overridden implementation
C. The compiler chooses randomly
D. No method can run

Answer: B. Dog's overridden implementation

Q8. [Java OOP] A class has private balance data and a public deposit() method that rejects negative amounts. Which OOP principle is demonstrated?

A. Encapsulation
B. Inheritance
C. Overloading
D. Serialization

Answer: A. Encapsulation

Q9. [Java I/O] An application must copy raw bytes from an image file. Which I/O family is the natural choice?

A. FileInputStream/FileOutputStream
B. PrintWriter only
C. Pattern/Matcher
D. LocalDate

Answer: A. FileInputStream/FileOutputStream

Q10. [Java I/O] A report program writes formatted text lines such as `Completed`. Which class is convenient?

A. FileInputStream
B. PrintWriter
C. FileChannel only
D. ZonedDateTime

Answer: B. PrintWriter

Q11. [Java I/O] A file reader should automatically close even when an exception occurs. Which construct is appropriate?

A. try-with-resources
B. switch
C. synchronized only
D. transient

Answer: A. try-with-resources

Q12. [Java Regex] An ID must contain exactly three uppercase letters followed by three digits. Which Matcher method is best for validating the whole input?

A. find()
B. matches()
C. next()
D. split()

Answer: B. matches()

Q13. [Java Date/Time] An application needs a meeting date and time interpreted in a specific region such as Asia/Kolkata. Which type fits?

A. LocalDate
B. LocalDateTime
C. ZonedDateTime
D. String only

Answer: C. ZonedDateTime

Q14. [Scala Basics] A Scala value must not be rebound after initialization. Which declaration should be used?

A. var
B. val
C. mutable
D. static

Answer: B. val

Q15. [Scala Basics] A developer has `val xs = List(2,3)` and evaluates `1 :: xs`. What happens to xs?

A. xs becomes mutable
B. xs is modified in place
C. xs remains unchanged and a new List is returned
D. xs is deleted

Answer: C. xs remains unchanged and a new List is returned

Q16. [Scala Basics] A program needs indexed elements that can be updated. Which structure from the handbook fits?

A. Immutable List
B. Array
C. Tuple
D. Option

Answer: B. Array

Q17. [Scala Pattern Matching] A status value can be 200, 404, or anything else. Which match pattern provides a fallback?

A. case null
B. case _
C. case else
D. default()

Answer: B. case _

Q18. [Scala Functions] A developer wants to double every value in List(1,2,3). Which operation expresses one output for each input?

A. map
B. flatMap
C. reduceByKey
D. collect

Answer: A. map

Q19. [Spark RDD] A developer has a small local Scala collection and wants to create an RDD for testing. Which API is appropriate?

A. sc.parallelize
B. sc.explain
C. groupByKey only
D. Files.walk

Answer: A. sc.parallelize

Q20. [Spark RDD] A Spark program calls `filter()` and `map()` but no action. What is the key behavior?

A. All results are immediately sent to the driver
B. Transformations are lazy and build lineage
C. Spark stops permanently
D. The RDD is automatically cached

Answer: B. Transformations are lazy and build lineage

Q21. [Spark RDD] A developer wants the number of records in an RDD. Which is an action?

A. map
B. filter
C. count
D. flatMap

Answer: C. count

Q22. [Spark RDD] A large RDD contains hundreds of GB. Which operation is risky because it brings all records to the driver?

A. filter
B. collect
C. map
D. persist

Answer: B. collect

Q23. [Spark RDD] A sales RDD contains `(region, amount)` pairs and the requirement is total sales per region. Which operation is directly suitable?

A. reduceByKey
B. take
C. collect only
D. mapPartitions only

Answer: A. reduceByKey

Q24. [Spark DataFrames] A team analyzes structured Parquet data with named columns and SQL-like filters and aggregations. Which API is generally preferred?

A. DataFrame
B. Raw byte stream
C. Socket source
D. FileInputStream

Answer: A. DataFrame

Q25. [Spark DataFrames] A production pipeline needs predictable column names and data types rather than guessing from input. What should it use?

A. Explicit schema
B. Random schema
C. No schema ever
D. Socket checkpoint

Answer: A. Explicit schema

Q26. [Advanced Spark] A developer wants to inspect how a DataFrame query will be executed, especially around joins and aggregations. Which method is useful?

A. df.explain()
B. df.sleep()
C. df.compile()
D. df.serialize()

Answer: A. df.explain()

Q27. [Advanced Spark] A DataFrame physical plan contains an `Exchange`. What does it commonly indicate?

A. Data redistribution/repartitioning, often a shuffle
B. A Java constructor call
C. A regex match
D. A checkpoint deletion

Answer: A. Data redistribution/repartitioning, often a shuffle

Q28. [Streaming] A new Spark streaming application should use DataFrame/Dataset-style structured APIs. Which API is generally preferred?

A. DStreams

B. Structured Streaming

C. FileInputStream

D. HashMap

Answer: B. Structured Streaming

Q29. [Concurrency/JDBC] A bank transfer must update two related database records and either both changes succeed or neither does. Which mechanism is required?

A. JDBC transaction with commit/rollback

B. ArrayList

C. Pattern.matches

D. map()

Answer: A. JDBC transaction with commit/rollback

Q30. [Concurrency/JDBC] A service repeatedly opens expensive database connections for short operations. What concept can reduce this overhead?

A. Connection pooling such as HikariCP

B. CopyOnWriteArrayList

C. flatMap

D. watermark

Answer: A. Connection pooling such as HikariCP

Medium – 30 Questions

Q31. [Java Collections] A team removes negative numbers while iterating through a List. Which approach best avoids modifying the collection independently during iteration?

A. Call list.remove() inside a for-each loop
B. Use Iterator.remove() after next()
C. Call clear() after every element
D. Use HashMap.put()

Answer: B. Use Iterator.remove() after next()

Q32. [Java Collections] A HashMap already contains key `A101` with value 5. The program executes `put("A101", 7)`. What is the result?

A. Two values are stored under the same key
B. The mapping for A101 becomes 7
C. The key is deleted
D. A101 is duplicated

Answer: B. The mapping for A101 becomes 7

Q33. [Java OOP] A subclass needs to call a parent constructor before executing its own constructor body. Which keyword is used?

A. this()
B. super()
C. parent()
D. base()

Answer: B. super()

Q34. [Java OOP] A class has `print(int)` and `print(String)`. A call passes an int. Which concept explains selecting the int version?

A. Runtime overriding
B. Compile-time overloading resolution
C. Dynamic field dispatch
D. Serialization

Answer: B. Compile-time overloading resolution

Q35. [Java OOP] A subclass tries to override a public parent method but declares the new method protected. What is the issue?

A. Visibility is reduced and overriding is invalid
B. It is valid because protected is always stronger
C. It becomes overloading
D. It becomes static

Answer: A. Visibility is reduced and overriding is invalid

Q36. [Java OOP] A Car contains an Engine object and uses it to perform work. Which relationship is modeled?

A. Inheritance
B. Composition
C. Overriding
D. Generic wildcard

Answer: B. Composition

Q37. [Java OOP] A developer wants closely related subclasses to share fields and implemented methods, while also requiring some methods to be implemented by subclasses. Which is a natural fit?

A. Abstract class
B. HashSet
C. Pattern
D. Array

Answer: A. Abstract class

Q38. [Java I/O] A program must search for `ABC123` inside a longer log line rather than require the whole line to match. Which Matcher method fits?

A. matches()
B. find()
C. compile()
D. equals()

Answer: B. find()

Q39. [Java Streams] A list of active users should be converted into a list of their names. Which pipeline is conceptually correct?

A. stream().map(User::getName).filter(User::isActive)
B. stream().filter(User::isActive).map(User::getName).toList()
C. stream().find().map(User::getName)
D. stream().reduce(User::getName)

Answer: B. stream().filter(User::isActive).map(User::getName).toList()

Q40. [Java Date/Time] A report stores only a due date and later adds seven days. Which type and operation are most appropriate?

A. LocalDate with plusDays(7)
B. ZonedDateTime with find()
C. Pattern with plusDays()
D. FileInputStream with plusDays()

Answer: A. LocalDate with plusDays(7)

Q41. [Scala Transformations] A dataset contains lines `"a b"` and `"c"`. The requirement is one flat collection of words. Which operation fits?

A. map(_.split(" "))
B. flatMap(_.split(" "))
C. filter(_.split(" "))
D. reduceByKey(_.split(" "))

Answer: B. flatMap(_.split(" "))

Q42. [Scala Transformations] A pipeline first removes inactive records and then expands each remaining record into its verified contacts. Which operation order is sensible?

A. filter(active) -> flatMap(contacts) -> filter(verified)
B. flatMap -> map -> filter(active) only
C. collect -> filter -> map
D. reduceByKey -> flatMap

Answer: A. filter(active) -> flatMap(contacts) -> filter(verified)

Q43. [Scala Functional] A function takes a value and a rule `A => Boolean` and returns only values passing the rule. What kind of function is rule?

A. Predicate
B. Constructor
C. Serializer
D. Window

Answer: A. Predicate

Q44. [Scala Functional] A tax function reads only `amount` and `rate` parameters and has no hidden global state. Why is it easier to test?

A. It is a pure function
B. It uses a socket
C. It mutates shared state
D. It forces a shuffle

Answer: A. It is a pure function

Q45. [Spark RDD] A developer calls `sc.textFile("/orders.txt")` to create an RDD from external text data. Which statement is correct?

A. It creates an RDD from an external text source
B. It creates only a local Scala List
C. It always collects all data to the driver
D. It creates a DataFrame automatically

Answer: A. It creates an RDD from an external text source

Q46. [Spark RDD] A team uses a transformation that can compute each output partition from a limited number of input partitions. What category is this?

A. Narrow transformation
B. Wide transformation only
C. Action
D. Checkpoint

Answer: A. Narrow transformation

Q47. [Spark RDD] An expensive cleaned RDD is used by three later actions. Which approach can avoid recomputing the entire lineage each time?

A. persist/cache the cleaned RDD
B. collect the RDD every time
C. convert it to a socket
D. call map() three times without reuse

Answer: A. persist/cache the cleaned RDD

Q48. [Spark RDD] A reused RDD may not fit completely in memory, but the team still wants cached data retained on disk when necessary. Which conceptual storage level is suitable?

A. MEMORY_AND_DISK
B. MEMORY_ONLY regardless of size
C. NO_STORAGE
D. DRIVER_ONLY

Answer: A. MEMORY_AND_DISK

Q49. [Spark RDD] A per-record operation creates an expensive client object. The same setup can be reused for all records in one partition. Which API is a conceptual fit?

A. mapPartitions
B. collect
C. take
D. count

Answer: A. mapPartitions

Q50. [Spark DataFrames] A CSV source sometimes contains malformed amount text. The team declares DecimalType. What should they understand?

A. DecimalType guarantees conversion of every string
B. Malformed input can still become null/corrupt or fail depending on reader configuration
C. DecimalType disables parsing
D. DecimalType automatically repairs all text

Answer: B. Malformed input can still become null/corrupt or fail depending on reader configuration

Q51. [Spark DataFrames] A team has structured data and mostly uses SQL-style filtering, joining and aggregation. Why might DataFrame be preferable to RDD?

A. DataFrame carries schema/column semantics and Spark can optimize structured operations

B. DataFrame cannot have a schema

C. RDD always performs better for structured data

D. DataFrame forces all data onto the driver

Answer: A. DataFrame carries schema/column semantics and Spark can optimize structured operations

Q52. [Advanced Spark] A DataFrame query has a selective filter that reduces rows from 100 million to 5 million before a custom expensive UDF. What is the likely performance benefit?

A. The UDF runs on fewer rows

B. The UDF becomes a JVM compiler

C. The filter forces all data to the driver

D. It removes the need for a Spark plan

Answer: A. The UDF runs on fewer rows

Q53. [Advanced Spark] A developer can implement a string cleanup using a built-in Spark SQL function or an opaque UDF. Which is generally preferred when equivalent?

A. Built-in function

B. UDF always

C. collect() first

D. Socket streaming

Answer: A. Built-in function

Q54. [Advanced Spark] A JDBC table is large and has a suitable numeric partition column. The team wants parallel reads. Which set of options is relevant?

A. partitionColumn, lowerBound, upperBound, numPartitions

B. watermark, window, slide, checkpoint

C. matches, find, Pattern, Matcher

D. val, var, tuple, guard

Answer: A. partitionColumn, lowerBound, upperBound, numPartitions

Q55. [Advanced Spark] A Parquet table has 50 columns, but a report needs only 4. What optimization can reduce unnecessary I/O?

A. Column pruning

B. Deadlock prevention

C. Autoboxing

D. DStream checkpointing

Answer: A. Column pruning

Q56. [Streaming] A streaming job must resume progress/state after a driver failure instead of starting from zero. What should the team configure?

A. Durable checkpoint location

B. Local temporary variable

C. CopyOnWriteArrayList

D. print()

Answer: A. Durable checkpoint location

Q57. [Streaming] A dashboard calculates a 30-minute rolling metric and updates it every 5 minutes. Which interpretation is correct?

A. Window length = 30 minutes; slide interval = 5 minutes

B. Window length = 5 minutes; slide = 30 minutes

C. Both are 30 minutes

D. Both are 5 minutes

Answer: A. Window length = 30 minutes; slide interval = 5 minutes

Q58. [Streaming] A live event arrives two minutes after its event time. Business users still want it included within a configured lateness allowance. Which mechanism addresses this?

A. Event-time processing with a suitable watermark

B. Only processing-time window with no event-time handling

C. HashMap

D. Java serialization

Answer: A. Event-time processing with a suitable watermark

Q59. [Concurrency] Two threads update a shared counter using a non-atomic read-modify-write sequence. Sometimes one increment disappears. What is the problem?

A. Race condition

B. Deadlock

C. Column pruning

D. Window slide

Answer: A. Race condition

Q60. [JDBC] A transfer updates Account A and Account B. The second update fails after the first succeeds. What should the program do if the operation must be all-or-nothing?

A. Roll back the transaction

B. Commit the first update immediately

C. Ignore the error

D. Close the JVM only

Answer: A. Roll back the transaction

Hard – 30 Questions

Q61. [Java Collections] A multithreaded service performs frequent concurrent reads and updates to a shared map. Replacing HashMap with which structure directly addresses the basic concurrent-access requirement?

A. ConcurrentHashMap
B. ArrayList
C. HashSet
D. ListIterator

Answer: A. ConcurrentHashMap

Q62. [Java Concurrency] Two threads can acquire locks X and Y. Thread 1 takes X then Y; Thread 2 takes Y then X. Both can wait forever. What is the clearest prevention strategy?

A. Use a consistent global lock-acquisition order
B. Add more mutable shared state
C. Call collect()
D. Replace Y with a List

Answer: A. Use a consistent global lock-acquisition order

Q63. [Java Concurrency] A listener registry is read thousands of times but modified only occasionally. Which collection is conceptually suited to safe iteration with expensive writes tolerated?

A. CopyOnWriteArrayList
B. HashMap
C. ArrayList with no synchronization
D. Stack

Answer: A. CopyOnWriteArrayList

Q64. [Java Concurrency] A service starts an asynchronous data load and needs to transform the result only after it completes, without blocking the initial submission. Which abstraction matches this requirement?

A. CompletableFuture
B. FileOutputStream
C. HashSet
D. Pattern

Answer: A. CompletableFuture

Q65. [JDBC] A transaction updates an order header and its payment row. The business rule says partial updates are unacceptable. Which sequence best matches the handbook?

A. setAutoCommit(false) -> execute updates -> commit on success -> rollback on failure
B. Commit after every statement
C. Enable auto-commit and never rollback
D. Use CopyOnWriteArrayList around SQL statements

Answer: A. setAutoCommit(false) -> execute updates -> commit on success -> rollback on failure

Q66. [JDBC] A high-throughput application repeatedly borrows short-lived JDBC connections. The team wants to reuse physical connections rather than create a new one for every operation. What is the correct concept?

A. Connection pooling such as HikariCP
B. Event-time watermark
C. RDD lineage
D. Regex find()

Answer: A. Connection pooling such as HikariCP

Q67. [Java Serialization] A class contains a cached token that should not be included in its default serialized state. Which declaration is appropriate?

A. private transient String cachedToken

B. private static String cachedToken

C. private final String cachedToken only

D. public String cachedToken

Answer: A. private transient String cachedToken

Q68. [Java OOP] A class currently calculates invoices, sends emails, and performs database backups. Changes to any of these areas require editing the same class. Which principle suggests restructuring this design?

A. Single Responsibility Principle

B. Autoboxing

C. Pattern matching

D. Type inference

Answer: A. Single Responsibility Principle

Q69. [Scala Functional] A pipeline processes millions of records. Only 10% satisfy a cheap condition, and a costly transformation is valid only for those records. What is the best functional strategy?

A. Filter early, then apply the expensive transformation

B. Apply the expensive transformation to all rows first

C. Collect everything to the driver first

D. Use flatMap regardless of output shape

Answer: A. Filter early, then apply the expensive transformation

Q70. [Scala Functional] A function has type `A => Boolean` and is passed to `filter`. What does that type tell you about its role?

A. It is a predicate deciding whether each A is retained

B. It transforms A into many records

C. It is a JDBC connection

D. It is a Spark action

Answer: A. It is a predicate deciding whether each A is retained

Q71. [Spark RDD] A developer writes `rdd.map(...).filter(...).map(...)` and then calls `count()`. Which statement best describes what happens?

A. Transformations build lazy lineage; count triggers execution

B. Every map immediately executes independently

C. filter always runs on the driver

D. count only changes the lineage and never executes

Answer: A. Transformations build lazy lineage; count triggers execution

Q72. [Spark RDD] A cluster loses one partition of an RDD derived from several transformations. No checkpoint exists, but the lineage is intact. What can Spark conceptually use to recover the lost partition?

A. Lineage to recompute the missing partition

B. HashMap keys

C. Matcher.find()

D. JDBC commit

Answer: A. Lineage to recompute the missing partition

Q73. [Spark RDD] A team wants total sales per region for a very large pair RDD. Which approach best avoids centralizing all records on the driver?

A. Map to (region, amount) and use reduceByKey(_ + _)
B. collect() and sum on the driver
C. take() all records
D. Convert every value to a String first

Answer: A. Map to (region, amount) and use reduceByKey(_ + _)

Q74. [Spark RDD] A pair-RDD join combines orders and customers by customer ID. The team sees a performance concern caused by data movement. Which explanation is most appropriate?

A. Key-based joins can require a shuffle/data redistribution
B. Joins always execute only on the driver
C. Joins never move data
D. Joins are Java constructors

Answer: A. Key-based joins can require a shuffle/data redistribution

Q75. [Spark RDD] An expensive RDD is reused across several actions, but caching everything in the application causes memory pressure. What is the best reasoning?

A. Persist only expensive intermediates whose reuse justifies storage
B. Cache every RDD automatically
C. Always collect before persisting
D. Never use persistence for reused data

Answer: A. Persist only expensive intermediates whose reuse justifies storage

Q76. [Spark DataFrames] A production pipeline receives CSV data from several teams. Column types occasionally change unexpectedly when inference is used. What design gives more predictable ingestion?

A. Define an explicit schema with expected names/types
B. Remove all schema information
C. Convert everything to bytes
D. Use socket streaming

Answer: A. Define an explicit schema with expected names/types

Q77. [Spark DataFrames] A team can solve a transformation using `lower()` and `regexp_replace()` or a custom UDF. The custom UDF is slower and opaque to Spark. What should the team choose?

A. Built-in Spark SQL functions
B. The UDF by default
C. collect() then Java processing
D. DStream checkpointing

Answer: A. Built-in Spark SQL functions

Q78. [Advanced Spark] A DataFrame aggregation's physical plan shows `Exchange`. The team asks why the query may be slower than a purely local operation. What is the best explanation?

A. Exchange commonly represents repartitioning/data movement, often a shuffle
B. Exchange means Java source compilation
C. Exchange means a regex failure
D. Exchange disables all Spark execution

Answer: A. Exchange commonly represents repartitioning/data movement, often a shuffle

Q79. [Advanced Spark] A performance engineer needs to understand whether a DataFrame join/aggregation introduces redistribution. Which diagnostic is most direct at handbook depth?

A. Inspect df.explain(true)
B. Call collect() and guess
C. Read serialVersionUID
D. Use Pattern.matches()

Answer: A. Inspect df.explain(true)

Q80. [Advanced Spark] A JDBC source has a suitable numeric column from 1 to 10,000,000. The team chooses an extremely high `numPartitions` without considering database capacity. What is the key concern?

A. Too many JDBC partitions can increase concurrent database connections/tasks beyond what the database supports

B. numPartitions changes column names

C. numPartitions is only a Java regex setting

D. numPartitions disables parallelism

Answer: A. Too many JDBC partitions can increase concurrent database connections/tasks beyond what the database supports

Q81. [Advanced Spark] A report reads a Parquet dataset partitioned by date and needs only one date plus three columns. Which combination best reduces I/O?

A. Filter on the partition column and select only required columns

B. Read every partition and every column, then filter on the driver

C. Convert Parquet to socket input first

D. Use collect() before filtering

Answer: A. Filter on the partition column and select only required columns

Q82. [Advanced Spark] A data platform receives flexible raw JSON, validates/transforms it, and serves repeated analytical queries. Which flow aligns with the handbook?

A. Ingest JSON -> validate/transform -> write efficient structured columnar output such as Parquet/Delta-style data

B. Convert everything to Java byte streams and never structure it

C. Use a socket source for durable production storage

D. Collect all JSON records to the driver

Answer: A. Ingest JSON -> validate/transform -> write efficient structured columnar output such as Parquet/Delta-style data

Q83. [Streaming] A new application needs DataFrame-style structured streaming with declarative operations. Which API is the intended choice over the legacy abstraction?

A. Structured Streaming

B. DStreams

C. FileOutputStream

D. HashMap

Answer: A. Structured Streaming

Q84. [Streaming] A file-streaming job receives new CSV files during the day. The team wants restart recovery and progress tracking. Which design is most appropriate?

A. Use a streaming file source with a durable checkpoint location

B. Use an ephemeral socket source and no checkpoint

C. Collect every file to the driver

D. Store progress only in a local variable

Answer: A. Use a streaming file source with a durable checkpoint location

Q85. [Streaming] A team wants a production source that can reliably replay data after failures. Which statement best matches the handbook's warning about sockets?

A. Socket sources are convenient demos but are not durable production sources

B. Socket sources are always the best durable replay mechanism

C. Socket sources automatically provide checkpoint recovery

D. Socket sources replace all file/message systems

Answer: A. Socket sources are convenient demos but are not durable production sources

Q86. [Streaming] A 10-minute event-time window must accept records up to 2 minutes late. The team considers increasing watermark tolerance to 10 minutes. What trade-off should they understand?

A. Longer tolerance can accept later data but may retain state longer

B. Longer tolerance always reduces state immediately

C. Watermarks only affect Java serialization

D. Watermarks remove the need for event time

Answer: A. Longer tolerance can accept later data but may retain state longer

Q87. [Streaming] A dashboard needs a rolling 30-minute metric recalculated every 5 minutes. A developer confuses the two values. Which interpretation is correct?

A. 30 minutes is window length; 5 minutes is slide interval

B. 5 minutes is window length; 30 minutes is slide interval

C. Both values are watermarks

D. Neither value is part of window semantics

Answer: A. 30 minutes is window length; 5 minutes is slide interval

Q88. [Concurrency] Two threads update shared state. The final value is sometimes correct and sometimes missing one update depending on scheduling. Which diagnosis is strongest, and why?

A. Race condition; the result depends on thread interleaving around a non-atomic read-modify-write

B. Deadlock; both threads must be waiting forever

C. Shuffle; data moved between partitions

D. Checkpoint failure; the JVM lost bytecode

Answer: A. Race condition; the result depends on thread interleaving around a non-atomic read-modify-write

Q89. [Concurrency] Thread A locks X then Y; Thread B locks Y then X. The system occasionally freezes. Which reasoning and fix best match the handbook?

A. Circular wait/deadlock risk; acquire multiple locks in a consistent global order

B. Race only; add more collections

C. UDF issue; replace the locks with a DataFrame

D. Watermark issue; increase lateness

Answer: A. Circular wait/deadlock risk; acquire multiple locks in a consistent global order

Q90. [JDBC] A transfer debits one account, then the credit fails. The database connection is using manual transaction control. What outcome is required to preserve all-or-nothing semantics?

A. Roll back the uncommitted debit

B. Commit the debit anyway

C. Re-run the credit without checking the debit

D. Close the pooled connection without rollback

Answer: A. Roll back the uncommitted debit